

DM128X Degree Project in Media Technology, First Cycle

Design and Evaluation of an LLM-Based Contextual UI Explanation Assistant for a Financial Operations Platform

Aren Seferi
KTH Royal Institute of Technology
Stockholm, Sweden
seferi@kth.se

Bachelor thesis report

ABSTRACT
This bachelor thesis report presents a project carried out in collaboration with Mrava.ai. The project investigates the design, implementation, and evaluation of a contextual UI explanation assistant, called Page Guide, for a financial operations platform. The feature aims to help users understand complex interface elements in the Accounts Payable area by combining drag-based UI exploration context, deterministic backend data, and large language model (LLM) generation. The work evolved from a frontend-oriented prototype into a backend-integrated architecture in which prompt construction and contextual enrichment are handled server-side. The implemented system supports an end-to-end streaming pipeline between frontend and backend, contextual explanation of selected interface regions, and deterministic backend support for selected labels, including *Blocked Approvals* and *Failed Invoices*. The report describes the motivation, related work, implementation, user evaluation, evaluation results, limitations, and future work.

Keywords
Large language models; intelligent user interfaces; contextual help; enterprise systems; financial operations platform; explainable assistance.

SAMMANFATTNING
Detta kandidatexamensarbete presenterar ett projekt som genomförts i samarbete med Mrava.ai. Projektet undersöker design, implementering och utvärdering av en kontextuell gränssnittsförklarande assistent, kallad Page Guide, för en plattform inom finansiella arbetsflöden. Funktionen syftar till att hjälpa användare att förstå komplexa gränssnittselement inom Accounts Payable genom att kombinera dragbaserad gränssnittsutforskning, deterministisk backenddata och generering med stora språkmodeller. Arbetet utvecklades från en frontend-orienterad prototyp till en backend-integrerad arkitektur där promptkonstruktion och kontextuell berikning hanteras på serversidan. Det implementerade systemet stödjer en end-to-end-strömmande pipeline mellan frontend och backend, kontextuell förklaring av valda gränssnittsregioner samt deterministiskt backendstöd för utvalda etiketter, inklusive *Blocked Approvals* och *Failed Invoices*. Rapporten beskriver motivation, relaterat arbete, implementering, användarutvärdering, utvärderingsresultat, begränsningar och framtida arbete.

Nyckelord
Stora språkmodeller; intelligenta användargränssnitt; kontextuell hjälp; företagssystem; finansiella arbetsflöden; förklarande assistans.

CCS Concepts
• Human-centered computing → Intelligent user interfaces; • Information systems → Enterprise applications; • Computing methodologies → Natural language processing.

1. INTRODUCTION

Modern enterprise platforms often contain dense, workflow-heavy interfaces that require users to interpret statuses, counters, actions, and domain-specific terminology very quickly. This is especially true in financial operations software and ERP-like environments, where users may work with invoice ingestion, approval workflows, bookkeeping, payments, and reconciliation in the same interface. Earlier work on enterprise-system usability and intelligent assistance has shown that even functionally capable systems can remain difficult to understand and learn in practice [1, 2, 3].

This work is carried out in collaboration with Mrava.ai, a financial operations platform with modules for Accounts Payable (AP), Accounts Receivable (AR), Treasury, analytics, document handling, and AI-supported workflows. The thesis does not attempt to redesign the whole product, nor does it focus on OCR model development or ERP data enrichment as its main research contribution. Instead, it focuses on one concrete problem: how an LLM-based assistant can help users understand complex UI regions within an existing financial platform. This places the work within a broader line of research on AI-based digital assistants, enterprise conversational agents, and intelligent user support in work-oriented software [4, 5, 6].

The implemented feature is called Page Guide. Its purpose is to let a user explore parts of the interface and receive a short contextual explanation of what the selected region means, why it matters, and what to do next. In the current prototype, the feature is scoped to the AP Supplier Invoices dashboard. The system combines drag-based UI exploration, prompt building on the backend, deterministic backend enrichment for selected labels, and server-sent event (SSE) streaming of the resulting explanation back to the frontend.

The purpose of this report is to describe the implemented prototype, the architectural decisions behind it, and the user evaluation conducted to examine whether Page Guide supports users in understanding the AP interface.

2. BACKGROUND AND RELATED WORK

Research on user assistance systems has long argued that digital systems should not only provide functionality but also help users understand how to use that functionality effectively. Earlier work on contextual help and intelligent assistance shows that support becomes more useful when it is tightly connected to the user interface and the user's immediate task context, rather than being separated into static manuals or generic help pages [2, 7, 8]. This is particularly relevant in workflow-oriented environments where users need explanations that are situated in the interface they are currently working in.

In enterprise settings, user support is especially important because misunderstandings can lead to delays, errors, and unresolved workflow issues. Prior work has discussed advanced user assistance systems and AI-based digital assistants in business environments, highlighting both their opportunities and their limitations [3, 6, 10]. Conversational agents and virtual assistants in organizations have also been studied from design and collaboration perspectives, showing that the practical usefulness of such assistants depends heavily on how they fit real work contexts [4, 5, 11].

At the same time, intelligent user interfaces raise questions of understandability and trust. Users need to understand what an intelligent system is doing, why it responds in a certain way, and whether the support is relevant in the current situation [9, 12]. Recent work has further suggested that large language models create new opportunities for natural and flexible user support embedded directly in interfaces [13, 14]. However, LLM-based systems also introduce a risk of hallucination or over-generalization when they are not grounded in reliable interface and backend context. That risk is particularly relevant in enterprise use, where perceived reliability and trust can directly affect whether users accept or reject algorithmic support [15].

This gap is especially relevant in financial operations platforms.

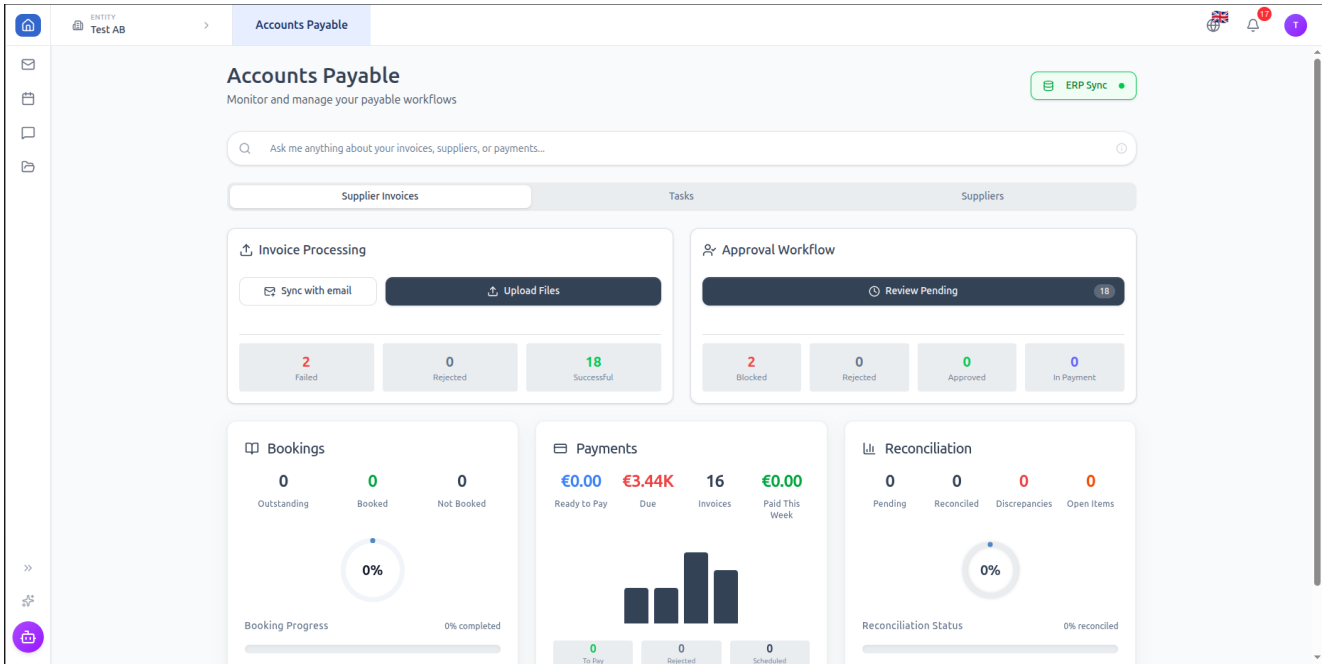


Figure 1: Page Guide visible in the Accounts Payable dashboard together with the draggable exploration bot.

Such systems contain specialized terminology, dense workflows, and tightly connected views that can be difficult to understand for both new and infrequent users. The setting therefore provides a relevant context for investigating whether LLM-generated contextual explanations can improve interface understanding in practice. It also motivates a central design question in this thesis: not only how to generate explanations, but how to combine interface context with deterministic backend knowledge so that the explanations remain both useful and trustworthy.

3. PROJECT CONTEXT AND PROBLEM DOMAIN

Within the Mrava platform, this thesis focuses on the AP area, specifically the Supplier Invoices dashboard, shown in Figure 1. This dashboard contains several tiles and sections related to invoice processing, approval workflows, bookkeeping, payments, and reconciliation. In usability terms, it can be understood as an enterprise work surface where multiple process states and action paths coexist in a compact interface, a context where interface-oriented assistance is especially relevant [1, 3].

The problem addressed in this thesis is that complex financial software interfaces are difficult to understand, while existing assistance is often too generic, too static, or too detached from the specific context in which the user needs help. A user may be able to see that there is one blocked approval, one failed invoice, or a particular reconciliation discrepancy, but not necessarily understand what that state means in the broader workflow or what action should be taken next. This kind of situated interpretive gap aligns with earlier work on contextual help and intelligent support in interactive systems [7, 8, 9].

The Page Guide feature was introduced to address this problem. The initial concept was a contextual assistant that observes what the user has explored in the interface and provides a short explanation in natural language. During implementation, however, it became clear that generic LLM explanations alone were not sufficient. The assistant needed to be grounded in actual system data for the explanation to become actionable. This led to the current hybrid architecture, where the UI indicates user focus, but the backend enriches the LLM prompt with deterministic business context before generation.

4. AIM AND RESEARCH QUESTIONS

The overall aim of this thesis is to design, implement, and evaluate a contextual UI explanation assistant for a financial operations platform. More specifically, the project investigates how an LLM-based assistant can be integrated into an existing enterprise system so that users can select or explore interface elements and receive short natural-language explanations grounded in the current workflow context.

The work is guided by the following research questions:

1. How can a contextual UI explanation assistant based on an LLM be designed and integrated into an existing financial operations platform?
2. How can deterministic backend context be combined with LLM generation in order to improve the correctness and usefulness of UI explanations?
3. To what extent can such an assistant help users understand complex interface elements in the Accounts Payable domain?

The first two questions are addressed through the design and implementation work described in this report. The third question is addressed through the user evaluation, which combines observation notes with questionnaire responses from participants who tested Page Guide on the Accounts Payable dashboard.

5. METHOD

The thesis follows an artifact-oriented and design-oriented methodology in which the central contribution is a working prototype integrated into an existing platform and evaluated with users. The work consisted of understanding the existing system, selecting a feasible feature scope, implementing the prototype iteratively, refining the architecture based on technical limitations, and evaluating the resulting assistant through observation and questionnaire data.

The process can be summarized in four main phases. First, the existing platform and repository were studied in order to understand the current frontend architecture, backend LLM infrastructure, and the user workflows in the AP area. This included identifying where UI context was already captured, how streaming LLM responses were handled, and how prompts were managed within the system.

Second, a minimum viable prototype (MVP) implementation was developed. The first version was frontend-heavy and reused an existing generic LLM streaming endpoint. This made it possible to get a working prototype quickly. However, the limitations of that version also became visible, especially in terms of generic explanations and insufficient grounding in real workflow data.

Third, the architecture was refined so that prompt construction moved to the backend and deterministic enrichment was introduced for a selected label. This refinement step is central to the thesis because it addresses a core risk in LLM-based support systems: superficially plausible explanations that are not sufficiently tied to actual system state [9, 12, 13].

Fourth, a small-scale user evaluation was conducted. Participants were shown the Accounts Payable dashboard, asked to interact with Page Guide, and then answered a questionnaire. During the sessions, observation notes were taken regarding dashboard understanding, interaction issues, explanation usefulness, and participant reactions. The final analysis combines qualitative observation notes with quantitative and open-ended questionnaire responses.

6. SYSTEM DESIGN AND ARCHITECTURE

6.1 Initial Frontend-Oriented MVP

The first working version of Page Guide was implemented by reusing several existing frontend and platform capabilities. The interface already contained a draggable bot that could move over UI regions, as shown in Figure 2. The bot could also store interaction records such as labels, pages, paths, timestamps, and counts in local storage. These capabilities made it possible to build a first version of the feature without introducing a new backend route immediately.

In that version, the Page Guide component read the bot context from local storage, built an inline prompt in the frontend, and sent a streaming request through the already existing LLM chat infrastructure. This proved the core concept and allowed a local MVP to be demonstrated quickly. However, it also revealed two key issues. First, the frontend owned too much prompt logic. Second, the explanations were often too generic when the LLM only received surface-level UI context.

6.2 Current End-to-End Architecture

The current implementation follows a cleaner and more platform-aligned architecture. The flow begins when the user drags the bot over relevant UI regions. The draggable bot identifies meaningful targets and stores interaction records in local storage. When the user opens Page Guide through the assistant UI, the frontend reads the stored context and builds a structured payload containing fields such as page identifier, current path, selected label, explored labels, context summary, and locale.

This payload is sent to a dedicated backend endpoint, `/v1/org/llm/chat/page-guide/stream`, using the existing tenant-aware authentication and SSE infrastructure. On the backend, the Page Guide service sanitizes the input, determines whether deterministic context can be resolved for the selected label, and then renders the appropriate prompt templates using the platform's prompt registry. If deterministic context is available, it is injected into the prompt before the LLM is called.

The result is streamed back through SSE using the same `content/done/error` event pattern as the platform's other LLM chat flows. The frontend reads the stream chunk by chunk and renders the explanation incrementally using markdown rendering.

This architecture is important because it turns the frontend into a thinner client. Instead of owning both context interpretation and prompt construction, the frontend mainly captures user focus and displays the streamed result. Prompt-building and context grounding are now backend responsibilities, which makes the design cleaner and more scalable. This also aligns with broader recommendations that intelligent assistance should be understandable, grounded, and embedded within the user's actual work context rather than acting as a detached generic chatbot [6, 9, 10].

6.3 Core Files and Responsibilities

Table 1 summarizes the core implementation files involved in the current prototype.

7. IMPLEMENTATION OF THE CURRENT PROTOTYPE

7.1 Context Capture and Request Building

The frontend Page Guide feature is currently scoped to the AP Supplier Invoices dashboard. The page identifier is hardcoded to `ap_supplier_invoices`, and the backend currently accepts only that page ID. From a product perspective, this is a deliberate limitation to keep the scope feasible. From an implementation perspective, it also makes it easier to design a stable end-to-end path before attempting broader rollout.

The drag-based context capture is central to the design. The bot does not simply behave as a floating icon; it actively identifies interface targets, stores interaction records, and allows Page Guide to infer what the user has recently explored. The selected label is currently taken from the most recently explored item, and that label plays a critical role in triggering deterministic backend support where available.

After the user requests an explanation, the response is shown in the Page Guide popover, as illustrated in Figure 3. This figure shows a general contextual explanation for the Invoice Processing module, generated from explored UI context and streamed back to the frontend.

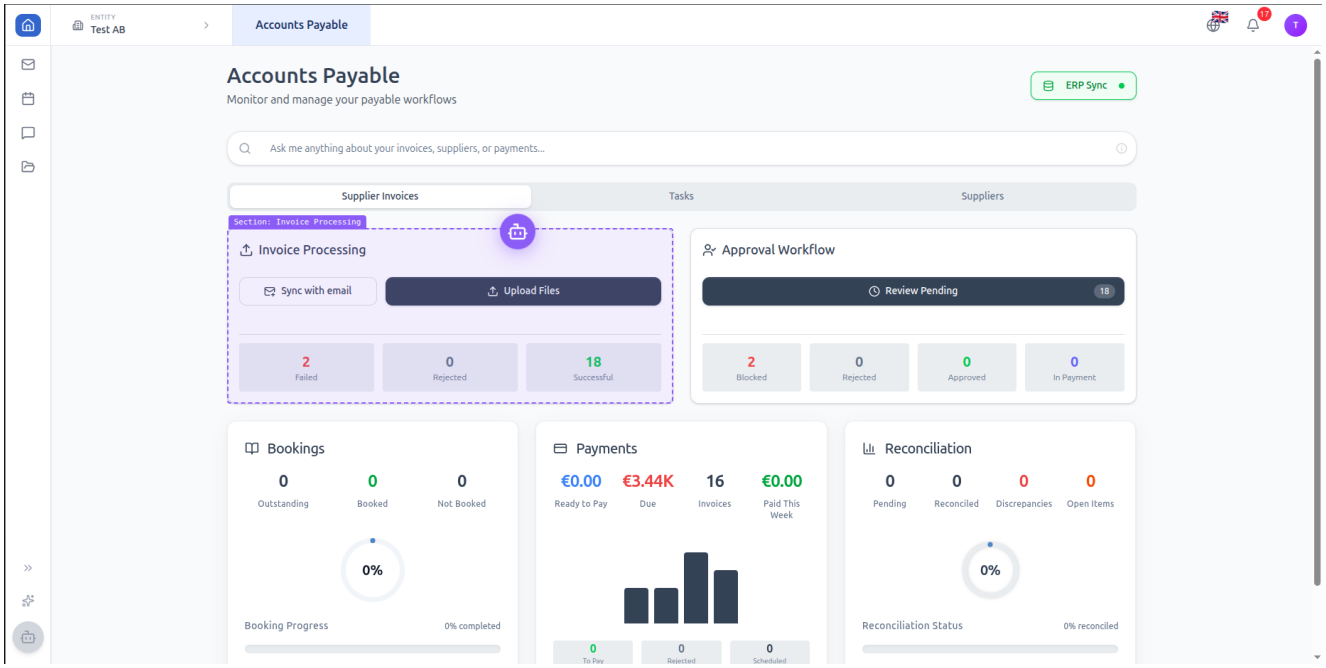


Figure 2: Drag-based exploration of the Invoice Processing region, where the bot highlights the selected UI area before an explanation is requested.

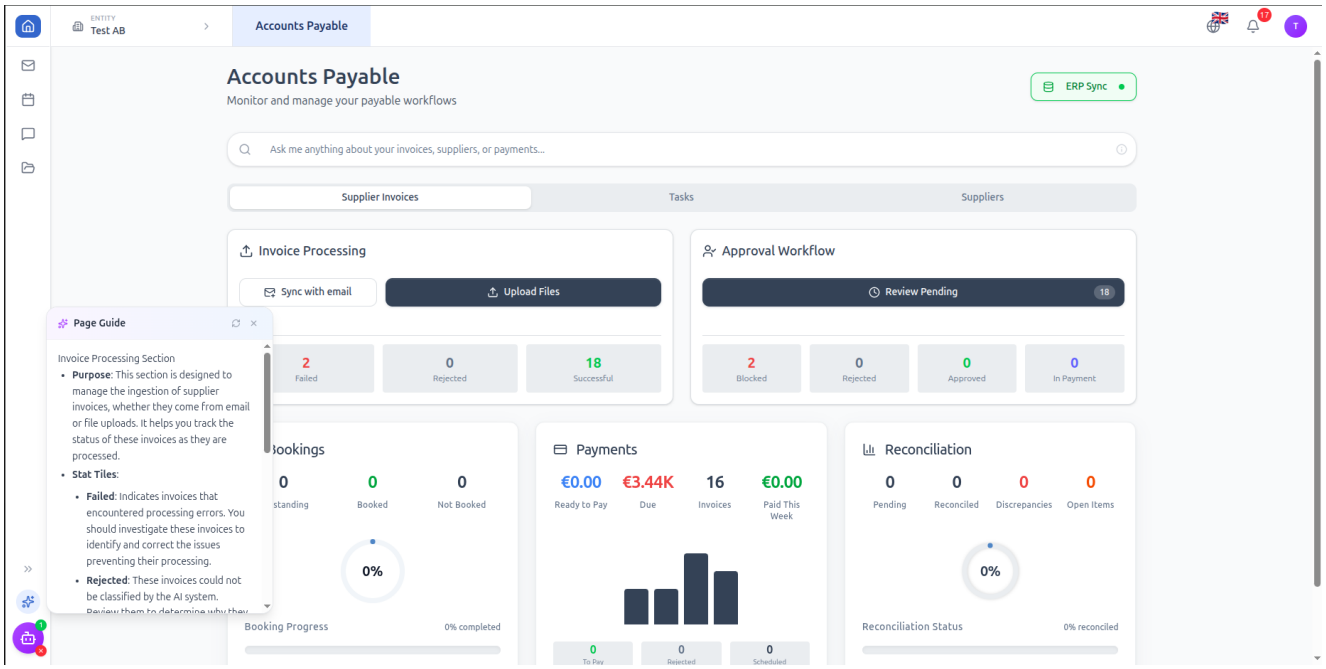


Figure 3: General contextual explanation for the Invoice Processing module, generated from explored UI context and streamed through Page Guide.

7.2 Prompt Construction and Server-Side Orchestration

One of the most important implementation decisions was to move prompt-building from the frontend to the backend. In the current architecture, the frontend no longer owns the system prompt or the logic that builds the final user message. Instead, the backend renders templates through the platform's prompt registry, which is consistent with how other LLM-driven capabilities in the Mrava platform are structured.

This decision improves maintainability and also strengthens the thesis contribution. It makes the Page Guide feature more than a frontend experiment: it becomes a feature aligned with the real product architecture. It also reduces the risk of prompt logic diverging across different clients or pages.

Table 1: Core files involved in the current Page Guide prototype.

File	Layer	Role in the implementation
mrava-portal/src/components/UIExplanationAssistant.tsx	FE	Main Page Guide popover UI; builds request payload and consumes streamed chunks.
mrava-portal/src/components/DraggableBot.tsx	FE	Captures explored UI context, labels, and persistence in local storage.
mrava-portal/src/components/Sidebar.tsx	FE	Mounts the Page Guide component and integrates it into the product shell.
mrava-portal/src/service/api/LLMChatService.ts	FE	Adds the `pageGuideStream()` call and client-side SSE handling.
mrava-portal/src/service/types/LLMChat.types.ts	FE	Defines the request shape and service contract for Page Guide.
mrava-portal/src/components/modules/02_approval_workflow.tsx	FE	Provides explicit `data-bot-context="Blocked Approvals"` label used by the resolver flow.
api/src/routes/tenant/shared/llm/chat.py	BE	Hosts the dedicated `/page-guide/stream` endpoint and SSE wrapper.
api/src/schema/agents.py	BE	Defines `PageGuideRequest` and validates incoming request data.
api/src/services/llm/page_guide_service.py	BE	Core logic: sanitization, label resolution, prompt rendering, deterministic enrichment, and stream orchestration.
api/src/tasks/contexts/prompts.py	BE	Contains the Page Guide system prompt and user template entries.
api/src/services/business/business_user_approval_filters.py	BE	Provides role-aware scoping for approval-related backend data.
api/src/services/business/supplier_invoice_lifecycle_queries.py	BE	Supplies shared query helpers for blocked-count and latest-status retrieval.

7.3 Deterministic Backend Support for Selected Labels

A key technical step in the implementation was the addition of deterministic backend support for the label *Blocked Approvals*. This label was selected as the first supported case because it is stable, meaningful to users, and tied to an already existing backend concept: invoices with the latest lifecycle status `P2_BLOCKED`.

When the selected label matches *Blocked Approvals*, the backend triggers a resolver that fetches verified data from the database. The data includes the blocked invoice count, scoped according to the current business-user visibility rules. In addition, if the current path contains an invoice item identifier, the backend can fetch item-level unresolved error context for that invoice. If no specific item is open, the backend can still retrieve a small sample of blocked invoices together with normalized error reasons and next-step hints. Figure 4 shows an example of a deterministically enriched Page Guide explanation for this label.

This means that the LLM no longer has to guess what *blocked* means. Instead, it receives verified context such as how many invoices are blocked, what type of issues they contain, and what action the user should consider next. The LLM's role becomes primarily linguistic rather than inferential.

Deterministic support was later extended to *Failed Invoices*. This resolver uses the latest lifecycle status `P1_FAILED` and injects verified backend context about failed invoice processing, including the failed invoice count, sampled failed invoice issues, and item-level issue information when available. This extension shows that the resolver pattern can be reused for additional Accounts Payable workflow labels beyond the first supported case.

This is closely aligned with the broader challenge identified in intelligent user interface research: users need support that is not only natural and flexible, but also understandable, relevant, and grounded in the actual state of the system [9, 12, 13].

7.4 Current Technical Status

The current prototype supports the complete frontend-to-backend streaming flow. Drag-based UI context capture works in the AP dashboard, the frontend can build and send a structured request, the backend can assemble prompts server-side, deterministic backend enrichment works for selected labels, and the final explanation is streamed back and rendered in the Page Guide popover.

Overall, the feature exists as a functioning prototype integrated into the actual platform, rather than as a separate design concept or isolated interface mock-up.

8. CURRENT LIMITATIONS

Although the current prototype is functional, it still has several limitations that are important both technically and scientifically.

First, deterministic support is currently implemented for only a small number of labels, mainly *Blocked Approvals* and *Failed Invoices*. Other labels still depend much more heavily on general contextual prompting. This means that the quality and specificity of explanations are not yet uniform across the interface.

Second, the resolver depends on label-text mapping. This is more fragile than using a canonical metadata identifier system and creates a risk of drift between frontend labels and backend resolver logic.

Third, the scope is intentionally narrow. The feature currently targets only the AP Supplier Invoices area, and the backend explicitly restricts page support accordingly. This is acceptable for the prototype stage, but it means that the current implementation cannot yet be treated as a product-wide assistant.

Fourth, the context capture relies partly on local storage history. This means that older or cross-page interactions may add noise if the history is not filtered carefully enough. In practice, this can influence the selected label and the final explanation.

Fifth, the current prototype still lacks several product-level elements, including dedicated automated tests for the Page Guide flow and an explicit in-product feedback loop for explanation quality.

Finally, the feature is not yet fully optimized for all contexts. For example, the draggable bot is disabled on mobile, and the current implementation does not include a dedicated abort mechanism for an in-flight stream when the popover is closed.

9. EVALUATION

9.1 Evaluation Design

The evaluation was designed as a small-scale user test of Page Guide on the Accounts Payable dashboard. The purpose was to examine whether the assistant helped users understand complex

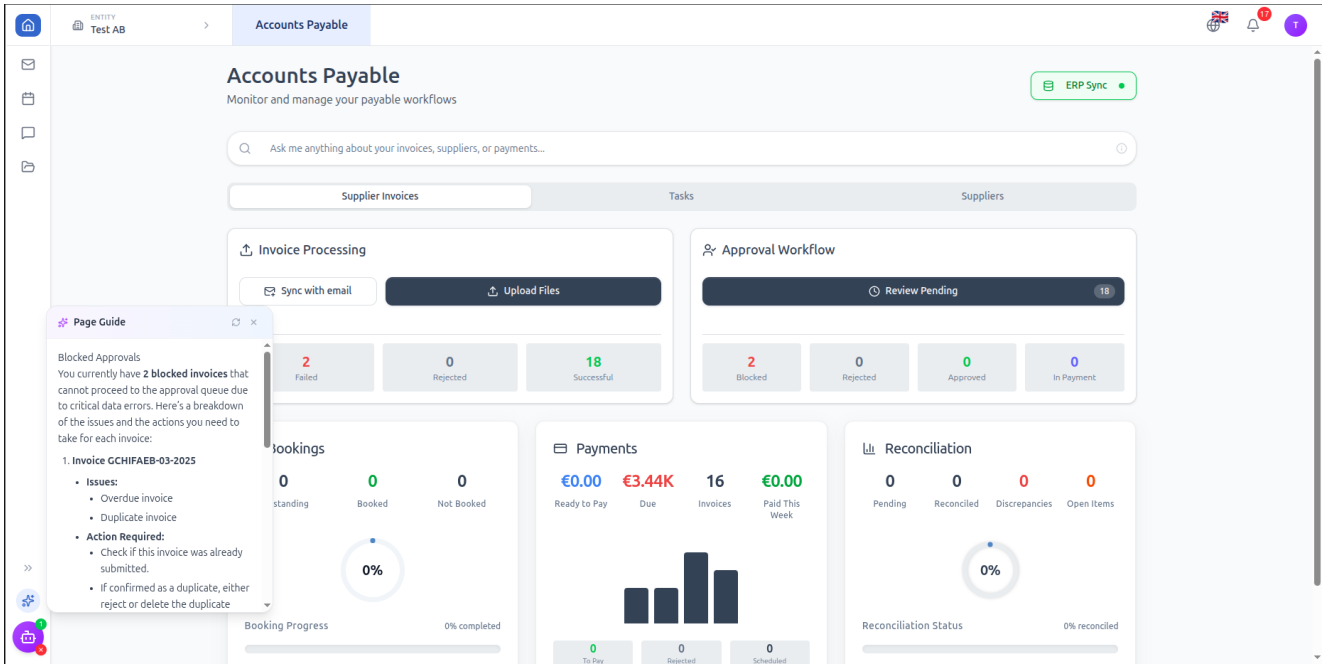


Figure 4: Deterministically enriched explanation for Blocked Approvals, including verified invoice-specific issues from the backend.

interface elements, financial workflow terminology, and possible next actions. The evaluation combined observation notes taken during the test sessions with a post-test questionnaire.

The evaluation focused on three main aspects. First, it examined participants' initial understanding of the Accounts Payable dashboard before using Page Guide. Second, it examined whether Page Guide explanations helped participants understand selected interface elements and workflow states. Third, it examined interaction issues, trust, perceived usefulness, and suggested improvements. This made it possible to evaluate both the explanation quality and the usability of the interaction model.

The test was not intended as a large statistical study. Instead, it was used as a formative evaluation of the prototype, with emphasis on identifying whether the assistant supported understanding and where the design still failed. The consent information, observation note structure, and questionnaire are included in the appendices to make the evaluation procedure more transparent.

9.2 Participants

Eight participants took part in the evaluation. All questionnaire respondents selected "Student" as their participant category. However, the participants differed in study background, finance familiarity, and previous understanding of invoice workflows. Most participants had limited or no experience with ERP systems or professional finance software, which made the sample relevant for evaluating Page Guide as support for non-expert and beginner users. Table 2 summarizes the participant profiles used in the analysis.

No names, email addresses, KTH IDs, or other direct identifiers were collected in the questionnaire. Participants are referred to as P1–P8 in the analysis. For internal analysis, the first questionnaire response was mapped to P1, the second to P2, and so on.

9.3 Procedure

Each session followed the same general structure. First, the participant was introduced to the purpose of the test and informed that participation was voluntary and anonymous. The participant was also informed that the session focused on evaluating the Page Guide prototype rather than the participant's own performance. The consent information used for the sessions is included in Appendix **??**.

The participant was then shown the Accounts Payable dashboard without using Page Guide, in order to observe their initial understanding of the interface. After this, the participant tested Page Guide by interacting with selected dashboard elements. The tested elements varied somewhat between participants, but included both general dashboard sections and more specific workflow states such as Blocked Approvals. The evaluator observed how the participant used the draggable bot, whether they understood how to request an explanation, and how they reacted to the generated answer. Finally, the participant completed a questionnaire.

The session structure allowed the evaluation to compare the participant's initial interpretation of the interface with their understanding after receiving Page Guide explanations.

9.4 Data Collection

Two types of data were collected. First, qualitative observation notes were written during each session. These notes focused on initial dashboard understanding, confusion points, interaction issues, explanation quality, deterministic explanations, perceived trust, and participant comments. The note-taking structure used during the sessions is included in Appendix **??**.

Second, a Google Forms questionnaire collected Likert-scale ratings and open-ended feedback after the test. The questionnaire included questions about finance familiarity, AI assistant familiarity, initial dashboard understanding, explanation clarity, explanation relevance, support for understanding the selected UI element,

Table 2: Overview of evaluation participants. Familiarity values use a 1–5 scale, where higher values indicate greater familiarity.

ID	Participant type	Finance familiarity	AI familiarity	Relevant note
P1	Engineering student	2	5	Beginner user with no ERP or invoice workflow experience.
P2	Business-related student	2	4	Had theoretical business knowledge but limited practical software experience.
P3	Sustainability master's student	1	4	Beginner user with no finance or ERP background.
P4	Media Technology student	2	4	Some finance knowledge, but no ERP experience; gave critical trust and usability feedback.
P5	Media Technology student	2	4	Some basic exposure to finance tools and receipt reporting, but not an accounting expert.
P6	Business student	4	5	More familiar with Accounts Payable, but no enterprise software experience.
P7	Media Technology student	2	4	Familiar with accounts payable, but expected more advanced assistant capabilities.
P8	Media Technology student	3	5	Limited invoice workflow experience; preferred action-based learning.

support for understanding next actions, trustworthiness, specificity, usefulness in a complex business or finance system, explanation length, and overall rating. The full questionnaire is included in Appendix **??**.

The open-ended questionnaire questions asked participants what they found most useful, whether anything was unclear or too generic, and what improvement they would suggest.

9.5 Analysis Method

The questionnaire responses were analyzed descriptively using mean, median, and range. Since the sample size was small, the quantitative results are interpreted as indicative rather than statistically generalizable. In this report, indicative means that the results are used to identify tendencies and possible design implications in the tested prototype, not to make claims about all potential users of financial operations platforms.

The observation notes and open-ended questionnaire answers were analyzed thematically. The analysis focused on recurring patterns across participants, especially initial dashboard understanding, usefulness of explanations, deterministic and data-specific explanations, interaction problems, trust, and suggested improvements. Participant-level findings were then compared with the questionnaire responses using the internal P1–P8 mapping.

10. RESULTS

10.1 Quantitative Questionnaire Results

The questionnaire results indicate that participants generally rated Page Guide positively after using it. As shown in Table 3, the overall rating of Page Guide had a mean of 4.25. The strongest scores were given for explanation relevance, usefulness in complex business or finance systems, support for understanding next actions, and trustworthiness. Initial understanding of the dashboard and status tiles was lower, which supports the need for contextual assistance in this type of interface.

The highest mean score was explanation relevance, with a mean of 4.88. This suggests that participants generally felt that Page Guide responded to the correct interface context. The assistant also scored highly on usefulness in a complex business or finance system and on helping users understand what action to take next, both with a mean of 4.50. Trustworthiness was also rated positively, with a mean of 4.38.

The lowest scores were for the questions about initial dashboard understanding before using Page Guide. The dashboard was rated with a mean of 3.00, and status tile understanding had a mean of 3.12. These two questions should not be treated as identical measures, but together they indicate that participants had only moderate initial understanding of the interface before receiving explanations.

The explanation length question had a mean of 3.38 on a scale where low values indicate too short or unclear and high values indicate too long or too detailed. This suggests that explanation length was generally acceptable, but qualitative comments show that some participants still wanted shorter or more adaptive explanations.

10.2 Tested Interface Elements

Participants tested several parts of the Accounts Payable dashboard, as shown in Table 4. The most frequently tested element was Blocked Approvals, followed by Invoice Processing and Reconciliation.

Blocked Approvals was especially important in the evaluation because it represented the clearest example of a data-specific explanation. The qualitative feedback on this is discussed further in Section 10.3.3.

10.3 Qualitative Findings

The qualitative findings are based on observation notes and open-ended questionnaire responses. The analysis identified five recurring themes across the eight evaluation sessions.

10.3.1 Initial dashboard understanding

Most participants understood the broad purpose of the Accounts Payable dashboard before using Page Guide. They generally recognized that the dashboard related to invoices, payments, approvals, and financial administration. However, this understanding was often shallow. Several participants struggled with domain-specific terms such as reconciliation, approval workflow, bookings, blocked approvals, and status tile meanings.

This distinction is important. The dashboard was not completely incomprehensible to participants, but many did not understand enough to know what specific workflow states meant or what actions they implied. For example, participants could often infer that invoices were being processed, but not necessarily understand why an invoice was blocked, what reconciliation involved, or how approval workflow states should be interpreted.

10.3.2 Usefulness of explanations

Page Guide was generally perceived as useful for explaining the dashboard more clearly than the interface alone. Participants reported that explanations helped them understand both the purpose of selected sections and the meaning of specific workflow states.

The most useful explanations were those that combined a short definition with practical next steps. Several participants reacted positively to explanations that described what an element meant and what action a user should take next. This was also reflected in the

Table 3: Descriptive questionnaire results from the Page Guide user evaluation.

Questionnaire question	Mean	Median	Range
Participant background			
Finance, accounting, ERP, or invoice workflow familiarity	2.25	2.0	1–4
AI assistant familiarity	4.38	4.0	4–5
Initial dashboard understanding			
Accounts Payable dashboard was easy to understand before Page Guide	3.00	3.0	2–4
Status tiles were understandable before Page Guide	3.12	3.0	2–5
Explanation quality and usefulness			
Page Guide explanation was easy to understand	3.88	4.0	3–5
Explanation was relevant to the tested UI element	4.88	5.0	4–5
Explanation helped explain what the UI element means	4.25	4.5	2–5
Explanation helped explain what action to take next	4.50	4.5	4–5
Explanation felt trustworthy	4.38	4.0	4–5
Explanation was specific rather than generic	4.00	4.0	3–5
Assistant would be useful in a complex business or finance system	4.50	5.0	3–5
Length of the explanation was appropriate	3.38	3.0	2–5
Overall assessment			
Overall rating of Page Guide	4.25	4.0	3–5

Table 4: Interface elements tested with Page Guide.

Interface element	Count
Invoice Processing	6
Failed Invoices	2
Rejected Invoices	1
Blocked Approvals	7
Approval Workflow	2
Payments	0
Reconciliation	4
Other	0

questionnaire results, where the question about understanding next actions received a mean score of 4.50.

However, the usefulness depended on explanation targeting. One participant wanted the term "Approval Workflow" itself explained, but felt that the assistant instead explained the fields below it. This shows that Page Guide must not only know the selected region, but also understand the user's intended level of focus: a term, a section, a tile, or a specific issue.

10.3.3 Deterministic and data-specific explanations

The strongest qualitative feedback concerned deterministic and data-specific explanations, especially Blocked Approvals. Participants valued explanations that referred to concrete invoice issues, reasons for blocking, missing information, overdue status, or next steps. These explanations were seen as more useful than general descriptions because they changed depending on the actual system context.

This pattern appeared across several participant notes. For example, P5 found the specific Blocked Approvals explanation useful, P6 found both reconciliation and Blocked Approvals explanations especially helpful, and P8 valued explanations that were specific and connected to actual system data. These findings suggest that Page Guide was most useful when it did not only define a concept, but also helped users understand what was currently wrong and how to proceed.

This supports the design decision to combine LLM-generated language with deterministic backend context. The evaluation primarily tested deterministic explanation behavior through Blocked Approvals. The implementation also supports deterministic backend enrichment for Failed Invoices, but the observation notes contain less evidence about how participants reacted to this label specifically. Therefore, the evaluation supports the value of deterministic grounding most clearly for Blocked Approvals, while Failed Invoices should be evaluated more directly in future work.

10.3.4 Interaction and usability issues

The most repeated usability issue was the interaction flow. Several participants did not immediately understand that the bot had to be dragged onto interface elements. Some expected to click the bot and chat with it like a normal chatbot. Others understood the dragging itself but did not notice or understand the generate button afterward.

The generate button was the clearest repeated issue, appearing in both observation notes and questionnaire responses. Participants suggested stronger visual feedback after selecting a section, such as making the button light up, flashing, adding a clearer prompt, or making the bot visually communicate that it can be moved.

The drag-and-drop concept was still liked by some participants once understood, but it was not self-explanatory enough for first-time use. This suggests that Page Guide needs onboarding, clearer affordances, or an alternative click-to-select interaction mode.

10.3.5 Trust and perceived reliability

Trust was generally positive in the qualitative feedback. Participants often connected trust to the fact that Page Guide felt embedded in the system rather than acting like an external chatbot.

However, trust was also conditional. One participant noted that trust in financial contexts depends partly on trust in the company or system behind the assistant. Another participant suggested that trust could be strengthened if Page Guide more clearly matched its explanations to actual system error messages. This supports the importance of deterministic grounding and transparent connection to backend data.

10.4 Participant-Level Summary

Table 5 summarizes the main qualitative findings from each participant.

10.5 Summary of Evaluation Results

Overall, the evaluation suggests that Page Guide helped participants understand the Accounts Payable dashboard, especially when explanations were connected to concrete workflow states and data-specific problems. Participants did not usually need help understanding that the dashboard was related to invoices and payments, but they did need help understanding more specific financial terminology, status tiles, and next actions.

Table 5: Participant-level qualitative summary.

ID	Main observation	Main implication
P1	Understood the broad dashboard purpose but struggled with finance terminology and needed help with drag-and-drop and generation.	Page Guide can support beginner users, but the interaction must be more self-explanatory.
P2	Already understood parts of the dashboard through business knowledge, but Page Guide helped connect theory to practical software actions.	Contextual explanations can still help users who have conceptual knowledge but limited system experience.
P3	Found the explanations easy to understand and valued the next-step structure, but wanted the explanation to remain visible while working.	Page Guide should support persistent guidance during task completion, not only one-time explanations.
P4	Valued unfamiliar and data-specific explanations, but criticized overly long or redundant explanations.	Explanation depth should adapt to user knowledge and avoid overexplaining obvious terms.
P5	Found the general overview and specific Blocked Approvals explanation useful, but expected a normal click-and-chat interaction.	The current drag model conflicts with some users' chatbot expectations and needs clearer affordances.
P6	Found reconciliation and Blocked Approvals explanations especially helpful and trusted the assistant, but found the generate step unclear.	The assistant can reduce confusion around financial concepts, but the request flow needs redesign.
P7	Valued specific contextual explanations and integration into the system, but wanted better terminology targeting and follow-up interaction.	Page Guide should allow more precise term-level explanations and follow-up questions.
P8	Trusted the assistant because it felt native to the app and valued data-specific explanations, but wanted shorter and more action-based guidance.	Future versions should move toward interactive, step-by-step guidance rather than only text explanations.

The strongest result is that data-specific explanations were more valuable than generic explanations. Blocked Approvals was repeatedly described as useful because it explained what was blocked, why it was blocked, and what could be done next. This supports the thesis argument that LLM-based UI assistance becomes more useful when combined with deterministic backend context.

The main weakness was not explanation relevance, but interaction design. The drag-and-drop model and generate button were not intuitive enough for first-time users. Several participants expected a more conventional chatbot interaction or wanted clearer visual feedback after selecting an element. Some participants also wanted shorter explanations, adaptive detail levels, clickable terms, follow-up questions, and step-by-step action guidance.

Therefore, the answer to the third research question is cautiously positive. Page Guide can help users understand complex Accounts Payable interface elements, particularly when explanations are contextual and data-grounded. However, the current prototype still needs clearer onboarding, improved interaction affordances, and more adaptive explanation behavior before it can be considered mature for real operational use.

10.6 Evaluation Limitations

The evaluation has several limitations. First, the sample size was small, with eight participants, so the quantitative results should be interpreted as indicative rather than statistically generalizable. The results can show tendencies in this prototype test, but they cannot establish how all potential users would experience Page Guide.

Second, all questionnaire respondents were students, which means the findings mainly reflect beginner and non-expert users rather than professional Accounts Payable users. This is useful for evaluating whether Page Guide helps users with limited domain knowledge, but it does not fully represent the needs, terminology familiarity, or workflow expectations of experienced finance professionals.

Third, the sessions were short and focused on a limited part of the platform. Participants mainly tested selected parts of the Accounts Payable dashboard, especially Blocked Approvals, Invoice Processing, and Reconciliation. This means that other dashboard areas, longer task flows, and repeated real-world use were not evaluated in depth.

Fourth, the evaluator both guided the sessions and took notes, which may have influenced participant behavior or the interpretation of observations. Since the notes were written during the sessions, some participant reactions may also have been summarized rather than captured as exact quotations.

Finally, deterministic backend explanations were evaluated most clearly for Blocked Approvals, while other deterministic labels, such as Failed Invoices, need more focused evaluation in future work.

11. DISCUSSION

One of the most important findings from the implementation and evaluation work is that a generic LLM explanation layer is not enough in a workflow-heavy enterprise environment. When the model only receives surface-level UI context, the explanation may still sound plausible but remain too generic to be practically useful. In financial operations software, this is a serious problem because users often need explanations that reflect the actual current state of the system rather than generic interface descriptions.

For this reason, the transition from a frontend-only MVP to a backend-enriched architecture should be seen as one of the main contributions of the work so far. The key contribution is not simply that an LLM can explain a tile or a workflow section, but that the explanation becomes significantly more useful when deterministic backend knowledge is injected before generation. This also responds to the wider challenge of making intelligent systems understandable to users, rather than merely impressive at text generation [9, 12].

This suggests a more general design principle for intelligent enterprise assistants. The user interface can provide a signal about user focus and interaction context, while deterministic domain logic should remain the source of truth for system state and workflow meaning. The LLM can then serve as a controlled language layer rather than as the main reasoning engine for business logic. In addition, making the assistant visibly tied to verified data may matter for user trust and acceptance, which connects to broader concerns about algorithm aversion and confidence in system outputs [15].

This hybrid model may have value beyond the specific Mrava case. It suggests that future enterprise explainability assistants should be designed not as purely conversational overlays, but as interfaces that combine interaction context, system knowledge, and language generation in a tightly integrated way [6, 10, 13].

12. CONCLUSION

This thesis has presented the design, implementation, and evaluation of Page Guide, a contextual UI explanation assistant for the Mrava financial operations platform. The work was scoped to the Accounts Payable Supplier Invoices dashboard and investigated how an LLM-based assistant can support users in understanding dense enterprise interface elements.

The first research question asked how a contextual UI explanation assistant based on an LLM can be designed and integrated into an existing financial operations platform. The project shows that such an assistant can be integrated by combining drag-based UI exploration, frontend context capture, a dedicated backend streaming endpoint, server-side prompt orchestration, and incremental SSE-based rendering in the frontend. This made Page Guide part of the existing platform architecture rather than a detached external chatbot.

The second research question asked how deterministic backend context can be combined with LLM generation to improve correctness and usefulness. The implementation shows that deterministic resolvers can enrich the LLM prompt with verified workflow data for selected labels such as *Blocked Approvals* and *Failed Invoices*. This shifts the LLM's role from guessing business meaning to explaining verified system state in natural language. The result is a more grounded explanation model where backend data acts as the source of truth and the LLM acts mainly as the language layer.

The third research question asked to what extent such an assistant can help users understand complex interface elements in the Accounts Payable domain. The evaluation suggests that Page Guide can support user understanding, especially when explanations are contextual, specific, and connected to concrete workflow problems. Participants rated explanation relevance, trustworthiness, usefulness in complex finance systems, and support for next actions positively. At the same time, the evaluation also showed that the interaction model needs improvement. The drag-and-drop flow and generate button were not self-explanatory enough for first-time users, and several participants wanted clearer affordances, shorter explanations, follow-up questions, or more action-oriented guidance.

Overall, the thesis shows that LLM-based UI assistance in enterprise software becomes more useful when it is grounded in deterministic backend context rather than relying only on generic interface descriptions. Page Guide therefore demonstrates a hybrid design direction for enterprise assistants: the interface captures user focus, backend logic provides verified workflow context, and the LLM transforms that context into understandable guidance. Future work should expand deterministic support to more labels, improve the interaction model, evaluate the feature with professional Accounts Payable users, and test the assistant in longer real-world workflows.

ACKNOWLEDGMENTS

The author would like to thank the industrial partner Mrava.ai for access to the platform, codebase, and architectural discussions that made this work possible.

REFERENCES

[1] Costa, C. J. 2010. Testing Usability of ERP Open Source Systems. In *Proceedings of the ACM SIGDOC European Chapter Conference: Crossing Design Boundaries*. DOI: https://doi.org/10.1145/1936755.1936763.

[2] Delisle, S. and Moulin, B. 2002. User Interfaces and Help Systems: From Helplessness to Intelligent Assistance. *Artificial Intelligence Review* 18, 117–157. DOI: https://doi.org/10.1023/A:1015179704819.

[3] Maedche, A., Morana, S., Schacht, S., Werth, D., and Krumeich, J. 2016. Advanced User Assistance Systems. *Business & Information Systems Engineering* 58, 5, 367–370. DOI: https://doi.org/10.1007/s12599-016-0444-2.

[4] Dhiman, H., Wächter, C., Fellmann, M., and Röcker, C. 2022. Intelligent Assistants: Conceptual Dimensions, Contextual Model, and Design Trends. *Business & Information Systems Engineering* 64, 5, 645–665. DOI: https://doi.org/10.1007/s12599-022-00743-1.

[5] Diederich, S., Brendel, A. B., and Kolbe, L. M. 2020. Designing Anthropomorphic Enterprise Conversational Agents. *Business & Information Systems Engineering* 62, 3, 193–209. DOI: https://doi.org/10.1007/s12599-020-00639-y.

[6] Maedche, A., Legner, C., Benlian, A., Berger, B., Gimpel, H., Hess, T., Hinz, O., Morana, S., and Söllner, M. 2019. AI-Based Digital Assistants: Opportunities, Threats, and Research Perspectives. *Business & Information Systems Engineering* 61, 4, 535–544. DOI: https://doi.org/10.1007/s12599-019-00600-8.

[7] Chilana, P. K., Ko, A. J., and Wobbrock, J. O. 2012. LemonAid: Selection-Based Crowdsourced Contextual Help for Web Applications. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 1549–1558. DOI: https://doi.org/10.1145/2207676.2208620.

[8] Yeh, T., Chang, T.-H., Xie, B., Walsh, G., Watkins, I., Wongsuphasawat, K., Huang, M., Davis, L. S., and Bederson, B. B. 2011. Creating Contextual Help for GUIs Using Screenshots. In *Proceedings of the 24th Annual ACM Symposium on User Interface Software and Technology*, 145–154. DOI: https://doi.org/10.1145/2047196.2047214.

[9] Eiband, M., Buschek, D., and Hussmann, H. 2021. How to Support Users in Understanding Intelligent Systems? Structuring the Discussion. In *Proceedings of the 26th International Conference on Intelligent User Interfaces*. DOI: https://doi.org/10.1145/3397481.3450694.

[10] Morana, S., Pfeiffer, J., and Adam, M. T. P. 2020. User Assistance for Intelligent Systems. *Business & Information Systems Engineering* 62, 6, 629–633. DOI: https://doi.org/10.1007/s12599-020-00640-5.

[11] Stieglitz, S., Mirbabaie, M., Möllmann, N. R. J., and Rzyski, J. 2022. Collaborating with Virtual Assistants in Organizations: Analyzing Social Loafing Tendencies and Responsibility Attribution. *Information Systems Frontiers* 24, 3, 745–770. DOI: https://doi.org/10.1007/s10796-021-10201-0.

[12] Völkel, S. T., Schneegass, C., Eiband, M., and Buschek, D. 2020. What is "Intelligent" in Intelligent User Interfaces? A Meta-Analysis of 25 Years of IUI. In *Proceedings of the 25th International Conference on Intelligent User Interfaces*, 477–487. DOI: https://doi.org/10.1145/3377325.3377500.

[13] Oelen, A. and Auer, S. 2024. Leveraging Large Language Models for Realizing Truly Intelligent User Interfaces. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. DOI: https://doi.org/10.1145/3613905.3650949.

[14] Pang, R. Y., Schroeder, H., Smith, K. S., Barocas, S., Xiao, Z., Tseng, E., and Bragg, D. 2025. Understanding the LLM-ification of CHI: Unpacking the Impact of LLMs at CHI through a Systematic Literature Review. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. DOI: https://doi.org/10.1145/3706598.3713726.

[15] Berger, B., Adam, M., Rühr, A., and Benlian, A. 2021. Watch Me Improve – Algorithm Aversion and Demonstrating the Ability to Learn. *Business & Information Systems Engineering* 63, 1, 55–68. DOI: https://doi.org/10.1007/s12599-020-00678-5.

TRITA-EECS-EX-2026:322
Stockholm, Sverige 2026

www.kth.se